

Feature-Level Insights into Artificial Text Detection with Sparse Autoencoders

Kristian Kuznetsov^{1,2}, Laida Kushnareva², Polina Druzhinina^{1,5}, Anton Razzhigaev^{1,5}, Anastasia Voznyuk³, Irina Piontkovskaya², Evgeny Burnaev^{1,5}, Serguei Barannikov^{1,4},

¹Skolkovo Institute of Science and Technology, ²AI Foundation and Algorithm Lab

³Moscow Institute of Physics and Technology, ⁴CNRS, Université Paris Cité, France

⁵Artificial Intelligence Research Institute (AIRI)

Abstract

Artificial Text Detection (ATD) is becoming increasingly important with the rise of advanced Large Language Models (LLMs). Despite numerous efforts, no single algorithm performs consistently well across different types of unseen text or guarantees effective generalization to new LLMs. Interpretability plays a crucial role in achieving this goal. In this study, we enhance ATD interpretability by using Sparse Autoencoders (SAE) to extract features from Gemma-2-2b’s residual stream. We identify both interpretable and efficient features, analyzing their semantics and relevance through domain- and model-specific statistics, a steering approach, and manual or LLM-based interpretation. Our methods offer valuable insights into how texts from various models differ from human-written content. We show that modern LLMs have a distinct writing style, especially in information-dense domains, even though they can produce human-like outputs with personalized prompts.

1 Introduction

The active development of large language models (LLMs) has led to the increasing presence of AI-generated text in various domains, including news, education, and scientific literature. Although these models have demonstrated impressive fluency and coherence, concerns about misinformation, plagiarism, and AI-generated disinformation have required the development of reliable artificial text detection (ATD) systems (Abdali et al., 2024). Existing ATD frameworks primarily rely on statistical measures, linguistic heuristics, and deep learning classifiers, yet these methods often lack interpretability, limiting their reliability in high-stakes applications (Yang et al., 2024).

A promising approach to enhancing interpretability in ATD is the use of Sparse Autoencoders (SAEs), which learn structured representations of textual data by enforcing sparsity constraints

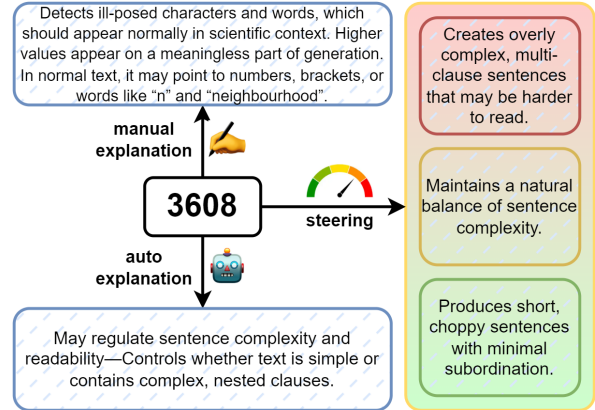


Figure 1: Interpretations of one of the most “universal” SAE features that are useful for ATD task.

(Huben et al., 2023; Makelov et al., 2024). We can extract human-interpretable features that capture the underlying structure of text.

In this study, we extend this line of research by applying SAEs from the Gemma-2-2b model (Team, 2024a) residual streams to analyze features that contribute to artificial text detection. By examining these features, we introduce a categorization of extracted features into discourse features (capturing long-range dependencies), noise features (highlighting unnatural artifacts), and style features (distinguishing stylistic variations). Our contributions are the following:

(i) we demonstrate the efficiency of SAE for the ATD task; (ii) we extract features which alone can effectively detect artificial texts for some domains and generation methods; (iii) interpreting these features, we identify meaningful patterns that contribute to ATD interpretability.

For our main dataset, we utilized a highly comprehensive and up-to-date dataset from GenAI Content Detection Task 1—a shared task on binary machine-generated text detection, conducted as part of the GenAI workshop at COLING 2025 (Wang et al., 2025). Hereafter referred to as the

COLING dataset, it contains a diverse range of model generations, from mT5 and OPT to GPT-4o and LLaMA-3. A complete list of models, along with generation examples, is provided in Appendix B.

We also performed additional experiments on the RAID dataset (Dugan et al., 2024), which contains generations from several models with various sampling methods and a wide range of attacks, from paraphrasing to homoglyph-based modifications. We provide the full list of models and attacks, along with examples of generations, in Appendix C.

2 Background

Given a token sequence $(t_1, t_2, \dots, t_n)$, an LLM computes hidden representations $\mathbf{x}_i \in \mathbb{R}^d$ at each layer l as $\mathbf{x}_i^{(l)} = g^{(l)}(\mathbf{x}_1^{(l-1)}, \mathbf{x}_2^{(l-1)}, \dots, \mathbf{x}_i^{(l-1)})$, where g represents a transformer block, typically including self-attention and feedforward operations. These activations encode meaningful information about text, but understanding models requires breaking them into analyzable features. Individual neurons are limited as features due to polysemanticity (Olah et al., 2020), meaning that models learn more semantic features than there are available dimensions in a layer; this situation is referred to as superposition (Elhage et al., 2022b). To recover these features, a Sparse Autoencoder (SAE) has been proposed to identify a set of directions in activation space such that each activation vector is a sparse linear combination of them (Sharkey et al., 2023).

Given activations $\mathbf{x}$ from a language model, a sparse autoencoder decomposes and reconstructs them using encoder and decoder functions with some activation function σ :

$$f(\mathbf{x}) = \sigma(\mathbf{W}_{\text{enc}}\mathbf{x} + \mathbf{b}_{\text{enc}})$$

$$\hat{\mathbf{x}}(f) = \mathbf{W}_{\text{dec}}f(\mathbf{x}) + \mathbf{b}_{\text{dec}}$$

for which $\hat{\mathbf{x}}(f(\mathbf{x}))$ should map back to $\mathbf{x}$. Here, the sparse and non-negative feature vector $f(\mathbf{x}) \in \mathbb{R}^M$ (with $M \gg d$) specifies how to combine columns of $\mathbf{W}_{\text{dec}}$ - learned features, or latents - to reconstruct $\mathbf{x}$.

3 Methods

In this work, we take a step towards improving the interpretability of artificial text detection using SAEs. We employ the Gemma-2-2B model along with pre-trained autoencoders on residual streams from Gemma-Scope (Lieberum et al., 2024).

Classifier models. For each even layer, we utilize an individual SAE ($f^{(l)}, \hat{\mathbf{x}}^{(l)}$) to extract learned features from each token. To obtain a feature vector $\mathbf{f}$ representing the entire text for layer l , we sum over all tokens, yielding

$$\mathbf{f} = \sum_{i=1}^n f^{(l)}(\mathbf{x}_i^{(l)})$$

We use an XGBoost classifier to evaluate the expressiveness of the full feature sets for each layer and identify the most important features for further analysis. The classifiers are trained exclusively on the Train subset of COLING and evaluated on the similar Dev set, as well as on the entirely distinct Devtest and Test subsets.

For a detailed feature analysis, we also use threshold classifiers on individual features.

Manual Interpretation and Feature Steering.

For manual interpretation, we analyzed the texts that activate the most important features. In layers with strong performance and generalization (layers 8 to 20), we selected the top 20 most significant features identified by XGBoost, as well as all features that achieved the highest detection performance for each domain and model using a threshold classifier. The selected features, their statistical properties, and example texts are publicly available¹.

To examine how learned features affect text generation, we use feature steering, which enables targeted modifications by selectively adjusting latent feature activations. For a given feature with number i associated with a specific text property, we first compute its maximum activation A_{max} across a reference dataset. During generation, hidden states are modified as

$$\mathbf{x}' = \mathbf{x} + \lambda A_{\text{max}} \mathbf{d}_i$$

where $\mathbf{x}$ is the original hidden state, $\mathbf{d}_i$ is the column of $\mathbf{W}_{\text{dec}}$ and λ is a scaling factor controlling the steering effect.

Furthermore, we employed the GPT-4-o model to analyze changes across all sequences and determine the nature or function of a particular hidden feature. (see Appendix G)

4 Results

General Detection Quality. To verify that SAE-derived features enable the detection of artificially

¹<https://mgtsaevis.github.io/mgt-sae-visualization/>

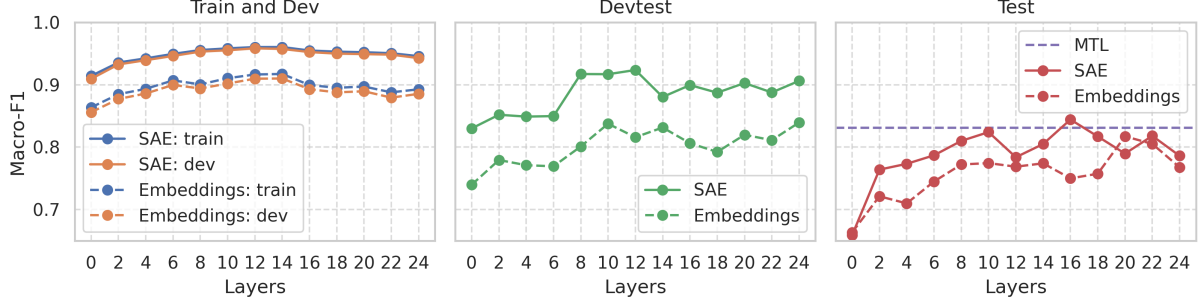


Figure 2: Macro F1 for XGBoost model on activations and SAE-derived features on different subsets of COLING

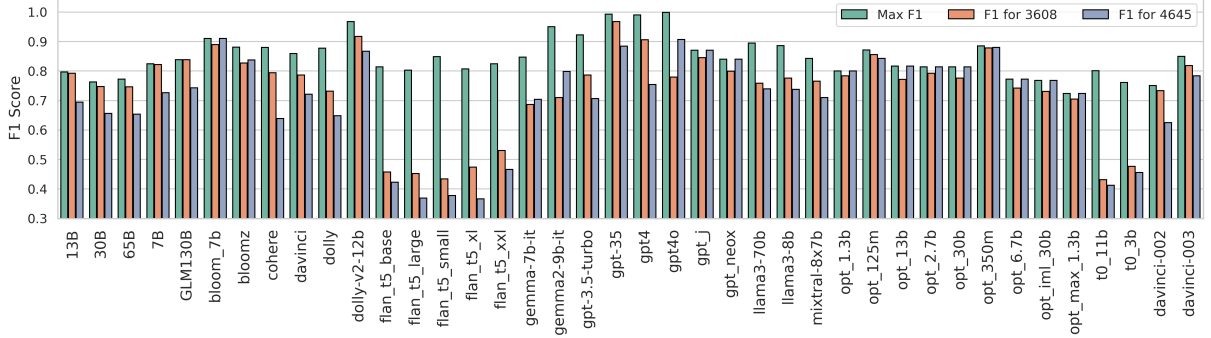


Figure 3: Macro F1 for a threshold classifier on individual features across each model for the 16th layer. Max F1 presents the maximum F1 score for every feature; features 3608 and 4645 are considered general features

generated texts, we apply XGBoost on these features and compare the results with XGBoost applied to mean-pooled activations from the layers. For training, we use the Train Subset, while testing is conducted on all remaining data.

As shown in Figure 2, both SAE features and activations perform well on this subset but degrade slightly on others. Notably, SAE features outperform activations both in training and across other subsets, suggesting that removing superposition helps the classifier focus on more fundamental, atomic features.

Although our primary objective is interpretability, it is worth noting that, at the 16th layer, SAE-derived features outperform the state-of-the-art MTL model on this dataset (Gritsai et al., 2025).

Domain/Model-Specific and General Features.

In our analysis of feature structure, we aim to distinguish between general features and domain- or model-specific features. Our focus is on the 16th layer, as its features have proven to be the most expressive and lead to the best generalization, as discussed in the previous section. Given the highly imbalanced distribution in the dataset, we split it into subsets by domains or models. Then we trained a threshold-based classifier for each feature across different subsets and analyzed their performance.

Interestingly, some features consistently exhibit high classification quality across multiple domains, which we refer to as general features. In contrast, other features are more specialized, performing well only within specific domains or detecting generations of a particular subset of models, highlighting their domain- or model-specific nature. Examples of these features and their performance are shown in Figure 4.

Some general features (e.g., 3608 and 4645 in layer 16) appear universal across domains and models. To demonstrate this, we compare the best feature for detecting each generator to these universal features (Figure 3). The graph shows that for older models (e.g., flan, t0), universal feature performance drops below random, while the *opt* family is the most "universal." This suggests distinct characteristics among model classes: older/weaker models (flan, t0), more advanced LLMs (opt, bloom, gpt_j, gpt_neo), and modern families (GPT-3.5+, LLaMa, Gemma). The next section explores these differences further.

Robust Feature Analysis. Building on Kuznetsov et al. (2024), we evaluate the classifier for the presence of harmful superficial features and those vulnerable to different types of attacks on artificial text classifiers, using the RAID dataset. Details